

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС» (НИТУ МИСИС)**

**ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)**

**КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)**

**Отчет по практике**

**Тема: «Realtime Chat App — веб-приложение для обмена  
сообщениями в реальном времени»**

**Студент: Фазлыев Эрик Маратович**

**Группа: БИВТ-24-2**

**Москва, 2026**

## Содержание

|                                 |    |
|---------------------------------|----|
| 1. Предметная область.....      | 3  |
| 2. Анализ требований .....      | 5  |
| 3. Этапы выполнения работы..... | 6  |
| 4. Структура базы данных.....   | 8  |
| 5. API и схемы .....            | 10 |
| 6. Выводы.....                  | 12 |
| 7. Источники .....              | 13 |

## **1. Предметная область**

### **Цель проекта**

Целью проекта являлась разработка клиент-серверного приложения, обеспечивающего регистрацию и авторизацию пользователей, создание личных и групповых чатов, обмен сообщениями в режиме реального времени, а также хранение и просмотр истории переписки.

Разрабатываемое приложение предназначено для обмена сообщениями между пользователями в режиме реального времени. Основными возможностями системы являются регистрация пользователей, создание личных и групповых чатов, хранение истории переписки, а также синхронизация сообщений между несколькими клиентами. Использование архитектуры клиент—сервер позволяет разделить ответственность между пользовательским интерфейсом, серверной логикой и системой хранения данных, обеспечивая удобство дальнейшего сопровождения и развития проекта.

В рамках распределения обязанностей автором настоящего отчёта была выполнена разработка серверной части приложения: проектирование архитектуры сервера, реализация REST API, WebSocket-сервера, механизма авторизации на основе JWT и слоя взаимодействия с базой данных на основе готовой схемы, спроектированной соисполнителем.

### **Стек технологий.**

Язык программирования Go — серверная часть; PostgreSQL — система управления базой данных (используется через слой репозитория); протокол WebSocket — обмен данными в реальном времени; JWT (access token, refresh token) — авторизация пользователей. Проектирование схемы базы данных, разработка миграций и развёртывание базы данных средствами Docker

Compose выполнены соисполнителем и в предметную область настоящего раздела не входят.

Выбранный стек технологий обусловлен требованиями к производительности и поддержке работы в режиме реального времени. Язык Go обеспечивает эффективную обработку большого количества одновременных подключений, PostgreSQL предоставляет надёжные средства хранения данных, а использование WebSocket позволяет минимизировать задержку при доставке сообщений между пользователями.

Разрабатываемое приложение построено по клиент-серверной архитектуре. Клиентская часть отвечает за пользовательский интерфейс и взаимодействие с пользователем, серверная часть реализует бизнес-логику, обработку запросов и обмен сообщениями в режиме реального времени, а система управления базой данных PostgreSQL обеспечивает хранение учетных записей пользователей, информации о чатах, сообщениях и токенах авторизации. Взаимодействие клиента с сервером осуществляется посредством REST API и протокола WebSocket, а сервер взаимодействует с базой данных через слой репозитория.

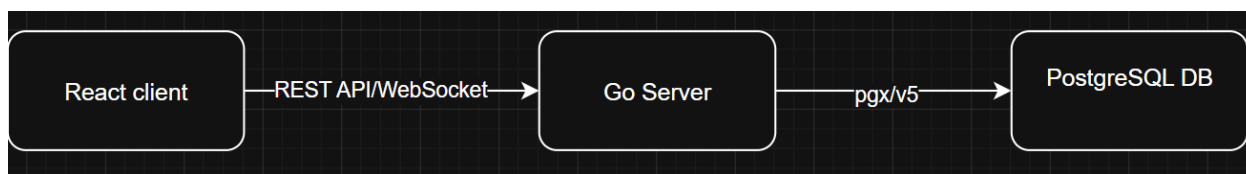


Рисунок 1.1 Архитектура приложения

## 2. Анализ требований

В ходе анализа требований были определены следующие функциональные задачи серверной части:

- проектирование архитектуры серверного приложения на основе многослойного подхода (транспортный слой — сервисный слой — слой доступа к данным);
- реализация REST API для операций аутентификации, управления чатами, участниками и сообщениями;
- реализация WebSocket-сервера для доставки сообщений в режиме реального времени;
- реализация механизма авторизации на основе JWT: выдача, проверка, обновление и аннулирование токенов доступа;
- реализация слоя доступа к данным на основе библиотеки pgx/v5 с использованием UUID в качестве первичных ключей, поверх уже развёрнутой базы данных;
- настройка компонентов промежуточного слоя (middleware) для контроля доступа к защищённым маршрутам и WebSocket-соединению.

Особое внимание при разработке серверной части уделялось безопасности и отказоустойчивости приложения. Для защиты пользовательских данных реализована авторизация на основе JWT, а использование многослойной архитектуры позволило разделить бизнес-логику, транспортный уровень и работу с базой данных, что упрощает сопровождение и дальнейшее развитие проекта.

### **3. Этапы выполнения работы**

1. Анализ функциональных требований и обоснование выбора технологического стека.
2. Проектирование структуры серверного приложения (пакеты `cmd/api`, `internal/config`, `internal/database`, `internal/models`, `internal/repository`, `internal/service`, `internal/transport/http`, `internal/transport/websocket`, `internal/security`, `internal/middleware`).
3. Реализация механизма авторизации: хеширование паролей, генерация и валидация JWT-токенов, реализация процедуры обновления токена.
4. Реализация REST API на основе сервисного слоя приложения, работающего с базой данных через слой репозитория.
5. Реализация WebSocket-сервера с проверкой токена доступа при установлении соединения и рассылкой сообщений участникам чата.
6. Проведение интеграционного тестирования совместно с клиентской частью и базой данных, выявление и устранение ошибок серверной части.

Разработка серверной части выполнялась последовательно — от анализа требований и проектирования архитектуры до реализации отдельных компонентов и их последующей интеграции. Такой подход позволил поэтапно проверить работоспособность каждого модуля и обеспечить корректное взаимодействие всех элементов системы.

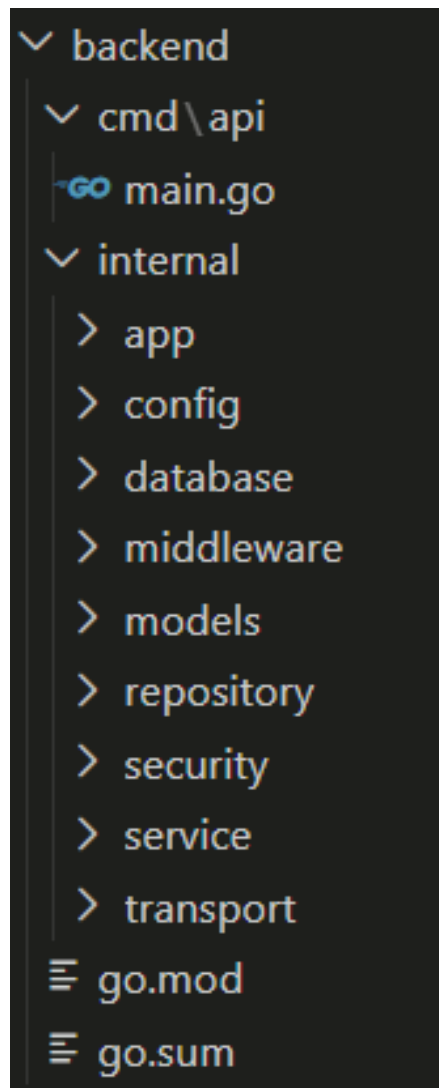


Рисунок 3.1 — Структура проекта backend

## 4. Структура базы данных

Серверная часть использует реляционную базу данных со следующей структурой (схема и развёртывание — зона ответственности соисполнителя, ниже приведена структура, с которой работает backend):

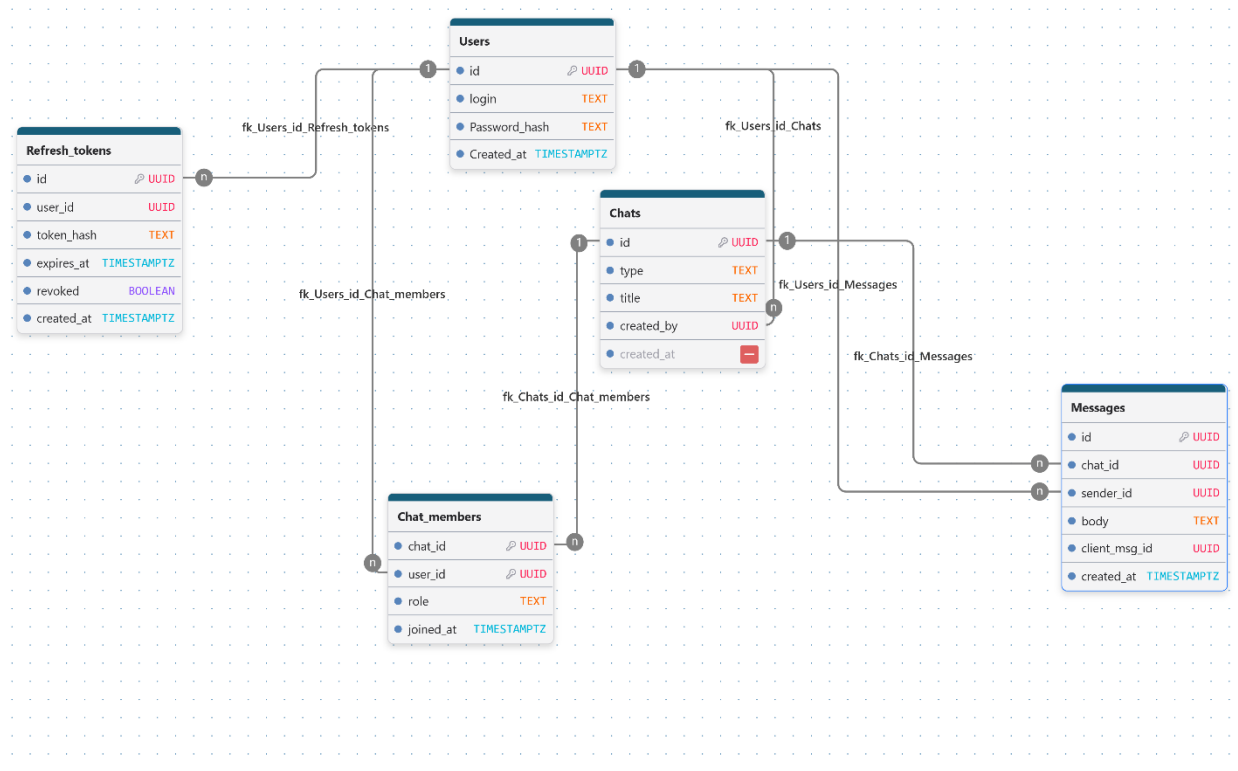


Рисунок 4.1 ER-диаграмма

Таблица users: id, login, password\_hash, created\_at — учётные записи пользователей.

Таблица refresh\_tokens: id, user\_id, token\_hash, expires\_at, created\_at — refresh-токены пользователей.

Таблица chats: id, type, title, created\_by, created\_at — личные и групповые чаты.

Таблица chat\_members: chat\_id, user\_id, role, joined\_at — участники чатов и их роли.



Таблица `messages`: `id`, `chat_id`, `sender_id`, `body`, `client_msg_id`, `seq`, `created_at` — сообщения.

Связи между таблицами: один пользователь может являться автором нескольких чатов; один пользователь может являться участником нескольких чатов; один чат может содержать несколько участников и сообщений; каждое сообщение принадлежит одному чату и отправлено одним пользователем; один пользователь может иметь несколько `refresh`-токенов.

Миграции базы данных: `001_users.sql` — создание таблицы пользователей; `002_chats.sql` — создание таблиц чатов и участников; `003_messages.sql` — создание таблицы сообщений; `004_refresh_tokens.sql` — создание таблицы `refresh`-токенов; `005_messages_seq.sql` — добавление порядковой нумерации сообщений; `indexes.sql` — индексы для ускорения запросов.

Развёртывание базы данных осуществляется средствами `Docker Compose` командой `docker compose up -d`, что обеспечивает запуск контейнера `PostgreSQL` и применение миграций.

## 5. API и схемы

Программный интерфейс, используемый клиентской частью приложения:

Таблица 5.1 Эндпоинты REST API

| Метод  | Эндпоинт                            | Описание                             |
|--------|-------------------------------------|--------------------------------------|
| POST   | /api/v1/auth/register               | регистрация пользователя             |
| POST   | /api/v1/auth/login                  | авторизация пользователя             |
| POST   | /api/v1/auth/refresh                | обновление access-токена             |
| POST   | /api/v1/auth/logout                 | завершение сеанса                    |
| GET    | /api/v1/chats                       | получение перечня чатов пользователя |
| POST   | /api/v1/chats                       | создание чата                        |
| GET    | /api/v1/chats/{id}/members          | получение перечня участников чата    |
| POST   | /api/v1/chats/{id}/members          | добавление участника                 |
| DELETE | /api/v1/chats/{id}/members/{userId} | исключение участника                 |
| DELETE | /api/v1/chats/{id}                  | удаление чата либо выход из него     |
| GET    | /api/v1/chats/{id}/messages         | получение истории сообщений          |
| GET    | /api/v1/messages/search             | поиск сообщений по тексту            |
| WS     | /ws                                 | WebSocket-соединение для             |

|  |  |                                             |
|--|--|---------------------------------------------|
|  |  | обмена<br>сообщениями в<br>реальном времени |
|--|--|---------------------------------------------|

При успешном выполнении запросов сервер возвращает коды состояния HTTP 200 (OK), 201 (Created) и 204 (No Content). При ошибках используются коды 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found) и 500 (Internal Server Error).

Схема взаимодействия клиентской части с сервером: после успешной авторизации клиент сохраняет access- и refresh-токены и передаёт access-токен в заголовке `Authorization: Bearer <token>` при обращении к защищённым маршрутам, а также при установлении WebSocket-соединения; при истечении срока действия access-токена клиент автоматически запрашивает новый через `/auth/refresh`; при выходе из системы токены удаляются на стороне клиента.

Схема отображения ролей на стороне интерфейса: для роли `admin` отображаются элементы управления добавлением и исключением участников группового чата; для роли `member` — только функции переписки и выхода из чата. Для личных чатов разграничение ролей не применяется.

## **6. Выводы**

По результатам прохождения практики спроектирована, развёрнута и наполнена миграциями база данных, а также разработана клиентская часть приложения, обеспечивающая взаимодействие с сервером посредством REST API и WebSocket-соединения. Реализованное решение демонстрирует применение принципов проектирования реляционных баз данных, контейнеризации инфраструктуры средствами Docker, построения клиентского приложения на основе React и TypeScript, а также организации обмена данными в реальном времени на стороне клиента, включая оптимистичное отображение сообщений и восстановление соединения при разрыве. Направления дальнейшего развития: индикация набора текста, статусы прочтения сообщений, передача файлов, отображение аватаров пользователей.

## **7. Источники**

1. React Documentation: официальный сайт. – URL: <https://react.dev> (дата обращения: 11.07.2026). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.
2. TypeScript Documentation: официальный сайт. – URL: [typescriptlang.org](https://typescriptlang.org) (дата обращения: 11.07.2026). – Текст: электронный.
3. Vite Documentation: официальный сайт. – URL: <https://vitejs.dev> (дата обращения: 11.07.2026). – Текст: электронный.
4. PostgreSQL Documentation: официальный сайт. – URL: [postgresql.org](https://www.postgresql.org) (дата обращения: 11.07.2026). – Текст: электронный.
5. Docker Compose Documentation: официальный сайт. – URL: [docker.com](https://docs.docker.com/compose/) (дата обращения: 11.07.2026). – Текст: электронный.
6. RFC 6455. The WebSocket Protocol / IETF; edited by I. Fette, A. Melnikov. – 2011. – URL: [ietf.org](https://tools.ietf.org/html/rfc6455) (дата обращения: 11.07.2026). – Текст: электронный.
7. RFC 7519. JSON Web Token (JWT) / IETF; edited by M. Jones, J. Bradley, N. Sakimura. – 2015. – URL: [ietf.org](https://tools.ietf.org/html/rfc7519) (дата обращения: 11.07.2026). – Текст: электронный.
8. Репозиторий проекта «realtime-chat-app» [Электронный ресурс]. – URL: <https://github.com/ByakuByaku/realtime-chat-app> (дата обращения: 16.07.2026).